Tobacco Leaf Disease Diagnosis

Adaptive Federated Learning with Client Selection and Gradient Compression

B.Tech project report: design, progress, results, limitations and future scope

Report date
9 November 2026

Project status
The web application, the baseline classifier and the four federated-learning experiments (A to D) are built, run and tested. The federated experiments were run once each (one random seed). Results are not yet imported into the web app, extra seeds have not been run, and behaviour on photos from other sources is untested.

Headline results
Federated experiments (one run each, 20 rounds, 8 simulated clients): Top-k compression cut the data uploaded by clients by **79%** (729 MB to 155 MB) and total traffic by **39%**, while test accuracy stayed within the noise of a 138-photo test set (81.9% without compression, 80.4% with it). Adaptive client selection showed **no consistent benefit** in this setup.

Centralised baseline (no federation), trained on 1,122 photos and scored on 138 separate photos it had never seen: 84.1% accuracy, macro-F1 0.729 (16 classes), roughly +/-6 points of uncertainty.

How to read this report. Sections 1 and 2 explain the project and its status in plain language. Sections 3 to 8 describe how it is built and tested. Sections 9 and 10 give the results, section 11 their reliability, and sections 12 to 14 the limitations, status and remaining work. A glossary is in the appendix.

Contents

1. Executive summary 3

2. Background and motivation 4

3. Dataset 5

4. System overview 6

5. Machine-learning design 8

6. Web application 10

7. Prediction service 11

8. Verification and testing 12

9. Results: the centralised baseline 13

10. Results: the federated experiments 16

11. Reliability: what the numbers can and cannot support 19

12. Limitations 19

13. Completion status 20

14. Future scope and remaining work 20

15. How to run and reproduce 22

16. References and credits 22

Appendix A. Glossary 24

Appendix B. The 16 classes in plain language 25

1. Executive summary

This project builds and studies a system that helps identify diseases on tobacco leaves from a photograph. It has two parts that are kept separate on purpose: a web application that people use, and a machine-learning pipeline in Python that trains and tests the models.

The research question is about federated learning. Normally, training a model means collecting all photos in one place. In federated learning the photos stay with their owners (here, simulated farms or labs) and only model updates travel to a central server. That protects data, but it costs network traffic and is complicated by the fact that different clients hold very different kinds of photos. The project tests two ideas that may reduce those costs: **adaptive client selection** (choose which clients train in each round using a score instead of luck) and **gradient compression** (send only the largest changes, called Top-k). Four experiments, A to D, compare every combination.

What was found. In the four runs, Top-k compression reduced the upload from 729 MB to 155 MB (-79%) and total traffic by 39% (the download from server to client stays full size). Accuracy with and without compression differed by less than the uncertainty of the test set. Adaptive selection did not give a consistent improvement. Each experiment was run once, so these are indications, not proof; section 11 explains what more would be needed.

Status at a glance

Part of the project	Status	Evidence
Dataset audit and preparation	Done	16 classes, leakage controls, synthetic images excluded from evaluation
Web application (9 pages)	Done	Runs locally; lint, type check and production build pass
Sign-in and database security rules	Done	10 automated rule tests pass on the Firebase emulator
Federated-learning code (FedAvg, selection, Top-k)	Done	24 Python tests pass; four full runs completed
Results import into Firebase	Done (tool)	Round-trip test passes with smoke data; real results not imported yet
Centralised baseline classifier	Done	84.1% accuracy, macro-F1 0.729 (test set, 138 photos)
Prediction service and diagnosis page	Done	Works locally with real uploads; a hosted version is prepared (section 7)
Federated experiments A to D on real data	Done, one seed	Section 10; each run scored once on the test set. Extra seeds not run
Real federated results shown in the web app	Not started	Results exist as files; importing them is the next step
Check on photos from other sources	Not started	Three informal spot checks all failed (section 12)
Deployment (Vercel)	Partly	Code and settings prepared and pushed to GitHub; live behaviour not verified in this report
Final report and version control	Partly	Code is on GitHub; the latest changes are not yet committed

Completion in one line. 8 of 12 work items are done, 2 are partly done and 2 are not started (a simple count, not a weighted measure of effort).

2. Background and motivation

Why tobacco leaf diseases matter. Tobacco is vulnerable to fungi, bacteria, viruses, insect pests, nematodes (microscopic soil worms) and environmental damage. Many of them show up first as spots, patches or colour changes on leaves. Recognising them early helps growers act sooner, but identifying a disease by eye takes experience and is easy to get wrong because several diseases look alike.

Why use a computer-vision model. A neural network can learn to recognise patterns in leaf photographs. The model used here, MobileNetV2, is small and fast, which makes it suitable for phones and simple servers. It is first trained on a large general photo collection (ImageNet) and then fine-tuned on tobacco leaves, a standard approach when there are only a few thousand training photos.

Why federated learning. Photos of crops may belong to different farms, cooperatives or research stations that do not want to hand over raw data. In federated learning, each client trains on its own photos and sends only the change to the model. The server combines those changes (the classic method is FedAvg, a weighted average) into a better shared model. Two practical problems follow:

- **Communication cost.** Every round, every selected client downloads and uploads a full model. Networks in rural areas can be slow or expensive.
- **Uneven data (non-IID).** One farm may see mostly one disease. Updates from such clients pull the shared model in different directions, which can slow learning.

The two ideas under test.

- **Adaptive client selection.** Instead of picking clients at random each round, give each client a score and pick the highest. In this project the score combines how badly the client did recently (50%), how long it has been waiting (30%) and how much data it holds (20%).
- **Gradient compression (Top-k).** After local training, a client sends only the largest 10% of its learned weight changes, plus their positions. The part it did not send is remembered and added to the next update (called error feedback), so little information is permanently lost.

Objectives

- Build a web application where signed-in users can upload a leaf photo, receive a clearly labelled model estimate and read results of experiments.
- Implement a federated-learning pipeline with FedAvg, random and adaptive client selection, and Top-k compression.
- Run four controlled experiments (A to D) that differ only in selection and compression, and compare accuracy, convergence speed and communication cost.
- Report results honestly, including limits, small-sample effects and anything that was not verified.

3. Dataset

The project uses the Tobacco Leaf Abnormality (TLA) dataset, published on Hugging Face as TamAko783/TPDD_Honglin_CLS. It has 16 classes and 1,571 images in total. The images were colour-normalised by the dataset authors with the Shades-of-Gray method.

Real versus synthetic images. Part of the training split consists of synthetic copies (images with _aug or _paste in their names). The experiments train on real photographs only, and the code asserts that no synthetic image can appear in the validation or test split. That leaves 1,122 training, 138 validation and 138 test photos (1,398 real photos in total; 173 synthetic training images are excluded).

Leakage controls. The dataset's own leakage-aware split is kept. Even so, four classes have a single validation photo and a single test photo, so the scores of those classes are extremely sensitive to one image (section 11).

Which model used which photos. The baseline in section 9.2 and all federated experiments use the 1,122 training photos. The model deployed on the website was afterwards retrained on all 1,398 real photos (section 9.4).

Real photos per class and split

Class	Train	Val	Test	Synthetic (not used)
anthracnose	33	1	1	27
black shank	7	1	1	53
brown spot	80	10	10	0
cmv	82	10	10	0
frog eye	80	10	10	0
genetic abnormality	17	1	1	43
healthy	80	10	10	0
nematodes	66	8	8	0
potato tuber moth	80	10	10	0
pvy	52	8	8	8
sunscald	108	14	14	0
target spot	80	10	10	0
tmv	103	14	14	0
tswv	18	1	1	42
weather fleck	94	12	12	0
wildfire	142	18	18	0
Total	**1122**	**138**	**138**	**173**

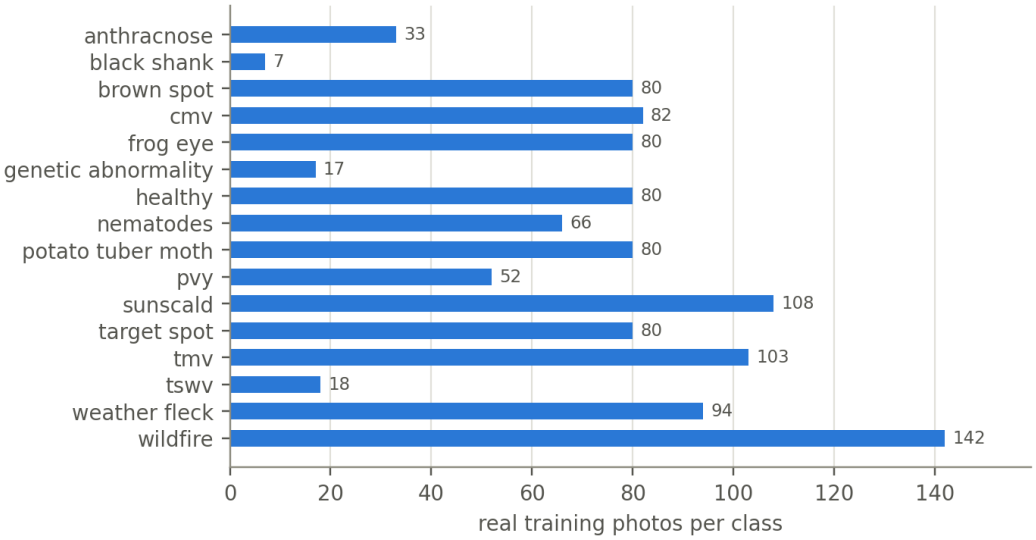


Figure 1. Real training photos per class. Classes are strongly unbalanced: wildfire has 142 photos, black shank has 7.

Licence and credit. The source repository states no explicit licence. It asks for academic use only, citation of the source papers (Lin et al., SPIE ICDIP 2022; Frontiers in Plant Science 2024) and no redistribution. For that reason the dataset is not stored in the project repository, in Firebase or on the hosting platform; it is downloaded on demand. The trained model file (not the images) is stored in the repository so the hosted prediction function can load it.

Datasets considered and not used. An earlier Kaggle tobacco dataset was a binary segmentation set, not a disease-classification set, and was dropped. A Roboflow classification dataset (CC BY 4.0, 1,268 images) has only three classes: alternaria alternata (the cause of brown spot), cercospora nicotianae (the cause of frogeye leaf spot) and a third class meaning neither of those is present. That third class is not the same as healthy, so merging it into the 16 classes would create wrong labels. It may be useful later as an external check for those two diseases. A search for further free tobacco disease datasets found none that is both freely downloadable and confirmed usable; the full 2,721-image source collection would require contacting its authors.

4. System overview

The system has a strict separation: training happens offline in Python. The website never trains anything; it displays results that were imported, and it sends a photo to a prediction service when a user asks for a diagnosis.

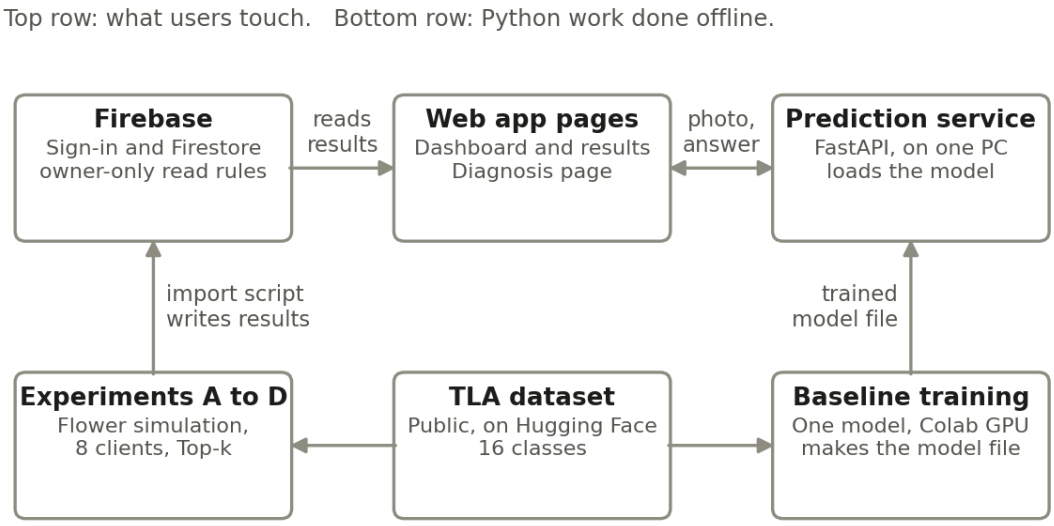


Figure 2. System components. The top row is what users touch; the bottom row is offline Python work.

Technology used

Layer	Tools
Website	Next.js (App Router), React, TypeScript, Tailwind CSS, Recharts for charts
Accounts and data	Firebase Authentication (email and password) and Firestore, protected by security rules
Model	PyTorch, MobileNetV2 pre-trained on ImageNet, fine-tuned on TLA
Federated learning	Flower 1.42 (simulation mode with the Ray backend), FedAvg, custom strategy
Prediction service	FastAPI. Locally: PyTorch model. Hosted: ONNX model on ONNX Runtime inside a Vercel Python function. No image is stored
Testing	pytest, the Firebase emulator with @firebase/rules-unit-testing, ESLint, TypeScript checks, ruff
Training hardware	Developer laptop (4 CPU cores, no GPU) for development; Google Colab for the baseline (GPU) and for the federated runs (CPU, see section 10.1)

5. Machine-learning design

5.1 Model and local training

Every client and the central baseline use the same MobileNetV2 network with a 16-way output layer. Local training uses stochastic gradient descent with momentum 0.9 and weight decay 1e-4, with random horizontal and vertical flips and small brightness changes as augmentation. The model's weights, including its BatchNorm statistics, are exchanged as one flat list of 32-bit numbers. A client's update is the weights after local training minus the weights it received.

5.2 One federated round

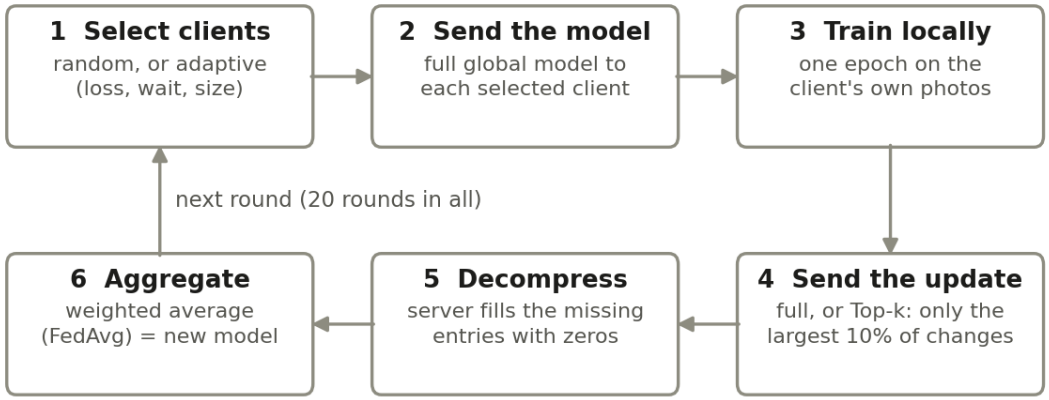


Figure 3. One federated round. The full cycle repeats for 20 rounds.

The server holds the shared model. In each round it chooses 4 of the 8 clients, sends them the current model, receives their updates and averages them, weighting each client by how many photos it has (FedAvg). After each round the new shared model is scored on the validation set; the test set is used once at the end, on the final shared model.

5.3 Data split between clients

The training photos are divided among 8 simulated clients with a Dirichlet distribution (alpha = 0.5). A small alpha makes each client's class mix uneven, which mimics real farms where one disease dominates. Each client keeps at least 20 photos (the clients ended up with between 53 and 283 photos). All four experiments use the same split and the same random seed (42), so differences between them come from the method, not from luck in the data.

5.4 Client selection

- **Random:** choose 4 of 8 clients uniformly at random each round.
- **Adaptive:** score each client as 0.5 x (its last training loss, scaled 0 to 1) + 0.3 x (rounds since it was last chosen, scaled) + 0.2 x (its data size, scaled), then pick the 4 highest. A client that has never been chosen counts as having the maximum loss, so every client gets a turn early. Ties go to the lower client number.

5.5 Top-k compression with error feedback

After local training, a compressed client keeps only the largest 10% of its **learned-weight** update entries (by absolute value) and sends their positions (4 bytes each) and values (4 bytes each): 8 bytes per kept entry instead of 4 bytes per entry for a full update. The entries it did not send are stored on the client as a residual and added to the next update, so they are delayed rather than lost. The server rebuilds a full-size update by filling the missing entries with zeros.

BatchNorm statistics are not compressed. The model also contains BatchNorm running statistics (means and variances, 34,112 of 2,278,480 numbers, 1.5% of the model). They are not gradients, and a variance can

never be negative. They are therefore always sent in full and their bytes are included in the communication cost. This rule was added after the first run of experiment C broke (section 10.6).

5.6 What is measured

- Accuracy and macro-F1 of the shared model on validation data after every round, and on the test set once at the end. Macro-F1 averages the F1 score of all 16 classes, so rare classes count as much as common ones.
- Communication in bytes. Downlink (server to client) is the full model times the number of selected clients. Uplink (client to server) is the real size of the arrays returned. Top-k reduces uplink only; the downlink stays full size.
- Convergence. The first round whose validation accuracy is within 0.01 of the run's best. The comparison script also reports rounds to a common target (95% of the lowest best accuracy among the runs), so runs are compared fairly.
- Per-class and per-client results and a confusion matrix (which diseases get mistaken for which).

5.7 The four experiments

Experiment	Client selection	Update compression	Question it answers
A: FedAvg baseline	Random	None	The reference every other run is compared with
B	Adaptive	None	Does smarter selection help on its own?
C	Random	Top-k (10%)	Does compression help on its own?
D	Adaptive	Top-k (10%)	Do the two ideas help together?

Shared settings: 8 clients, 4 chosen per round, 20 rounds, 1 local epoch, learning rate 0.01, batch size 32, images resized to 160 px, seed 42, basic augmentation (the stronger recipe used for the baseline was not used here).

6. Web application

The website is built with Next.js and works in light and dark themes. Pages inside the application require sign-in; the landing, login and register pages are public.

Page	Purpose
Landing	Explains the project with a diagram of one server and several clients, the four-step federated round, the 16 recognisable conditions, a three-step guide, and an honest status section
Login and Register	Email and password sign-in with validation and readable error messages
Dashboard	Welcome banner with a button to the diagnosis page, summary boxes, the signed-in user's own recent diagnoses (label, confidence, date), a project-progress checklist that ticks from real data, and recent experiments
Experiments and experiment detail	List of runs; detail page shows rounds, per-class metrics (rare classes flagged), per-client results and a shaded confusion matrix. When empty, it shows the planned design of experiments A to D and the three commands that bring results in
Results	Compare chosen experiments: accuracy, macro-F1, training and validation loss, final precision, recall and F1, cumulative communication, client participation. Every chart has a table view and each experiment keeps one fixed colour. When empty, it lists the charts that will appear
Diagnosis	Upload a leaf photo (large photos are shrunk in the browser before sending), see the predicted disease, confidence, top three guesses, a plain-language explanation and a guide to all 16 classes
About	Problem, objectives, workflow, selection method, Top-k, the A to D table and limitations
404 and error pages	Friendly fallbacks

6.1 Honesty rules built into the interface

- Missing values are shown as an em dash or "No data yet", never as zero.
- With no results imported, the pages say so; no placeholder numbers are shown. The explanatory text on the empty pages describes the plan only, with no figures.
- When no prediction service is connected, the diagnosis page shows a clearly labelled placeholder and disables the button; it never invents a prediction.
- Every result carries the notice that it is a model estimate and not a professional diagnosis, and results below 50% confidence get an extra low-confidence warning.
- The landing page's status section quotes the 84.1% baseline with its uncertainty and summarises the federated findings as single-run results; it carries no invented numbers.

6.2 Data model and security

Each signed-in user owns a private area of the database: experiments with their per-round metrics, and a history of predictions. The security rules, enforced by the database itself and not only by the website, allow each user to read only their own data, deny every client write to experiments and rounds (only the trusted Python import script may write them, using an administrator credential kept outside the code), allow a user to create a prediction record only if it has the expected fields and a server timestamp, and deny everything else by default. Ten automated tests check these rules against the Firebase emulator.

Privacy. A prediction record stores only the predicted label, the confidence, the model version and a timestamp. The photo itself is never stored: it is held in the browser, sent to the prediction service in memory and discarded. No passwords or keys appear in the source code; real configuration lives in an ignored .env.local file, and an .env.example shows the format.

6.3 Plain-language disease guide

Many class names are abbreviations (TSWV, TMV, PVY, CMV). The result card therefore shows the full name, the cause (fungus, virus, bacterium, water mould, pest, environment or genetic), the organism where one exists, a one-line summary and what the disease usually looks like. A guide on the same page describes all 16 classes. Several virus names mention another crop (for example tomato spotted wilt virus) only because it was

first described there; they also infect tobacco. These descriptions were written from general plant-pathology knowledge and have not yet been reviewed by an expert; they are background, not treatment advice.

7. Prediction service

The prediction service answers one question: given a photo, which class is it? The website sends the photo as a form upload and receives the label, the confidence, the three most likely classes and the model version. It exists in two forms that use the same preparation code, which a test checks byte for byte:

- **Local service** (FastAPI, PyTorch model) for development on the owner's computer.
- **Hosted function** (FastAPI and ONNX Runtime inside a Vercel Python function, no PyTorch), so the deployed website can diagnose without a separate server. The ONNX model gave the same predicted class as the PyTorch model on all 138 test photos (largest numerical difference about 5e-6). Vercel limits request bodies to 4.5 MB, so the page shrinks large photos before sending them. Whether the function builds and responds on Vercel itself, including start-up time, was not verified in this report.

Model behind it. The model in the repository is the one retrained on all 1,398 real photos (section 9.4). It has no held-out score of its own.

- **Input checks.** Only JPEG, PNG or WebP; empty or corrupt files and files over the size limit are rejected with clear error codes (415, 413, 400). All of these were exercised against the hosted function's code with the new model.
- **Preparation.** The photo is colour-balanced with the Shades-of-Gray method (power 6) and resized to the model's size. This approximates the dataset authors' processing, whose exact settings were not published. A check on the test photos showed the extra step changes accuracy by about 1.5 points (84.1% versus 82.6%); it is not the main source of error on outside photos.
- **Safety.** The local service refuses to start without a trained model, so it can never answer from random weights. Its automated tests use a throwaway random model and check behaviour (types, limits, response shape), not accuracy.

8. Verification and testing

The table lists what was actually checked, and how. Items marked as not verified should not be assumed to work.

Area	Check	Result
Security rules	10 tests on the Firebase emulator (owner reads, denied writes, validated prediction create, default deny)	Pass (re-run in the final check)
Results import	Python import into the emulator, then the web app's own parser reads it back (2 tests)	Pass, with smoke data
Python pipeline	24 tests: partitioning, Top-k round trip, split between weights and BatchNorm statistics, selectors, metrics, strong augmentation, CPU switch, prediction service	Pass
Code quality	ruff scan for unused code, ESLint and TypeScript checks, production build of all 11 pages	Pass; one unused import removed
Hosted function code	Health check, valid photo, wrong type (415), corrupt (400), empty (400), over 4 MB (413), missing file (422), run locally with the deployed model	Pass locally; not verified on Vercel
ONNX conversion	All 138 test photos through both the PyTorch and ONNX models	Same class on 138 of 138
Federated runs	A, B, C and D at full settings, 20 rounds, on Colab CPU; all four finished with finite numbers	Complete, one seed
Experiment C, first attempt	Global model became NaN at round 13; found and fixed (section 10.6); rerun finished normally	Fixed
GPU path for federated runs	On the Colab GPU image used, client processes crashed inside PyTorch's CUDA dropout step (a floating-point exception). A CPU switch avoids it. The cause was not isolated	Does not work on that image
Charts with real data	Rendering of real federated results in the web app	Not verified (results not imported yet)
Visual review by a person	Reading every page in a browser	Not done

8.1 Pipeline smoke tests. Before the real runs, experiments A, B and D were run for a handful of rounds on tiny settings, and again on Colab, to prove the pipeline runs end to end. Their accuracy (about 13%) is meaningless. The real runs in section 10 replace them.

9. Results: the centralised baseline

A conventional classifier trained on all training photos in one place, with no federation. It serves two purposes: it provides the model behind the website's diagnosis page, and it gives a reference point for the federated runs. **Its numbers are not directly comparable with the federated results in section 10**, because it used a different training recipe, a larger image size and a longer schedule.

9.1 Two versions of the baseline

The first baseline used basic training (constant learning rate 0.01, 160 px images, 15 epochs). Its validation score swung widely from epoch to epoch (Figure 4), a sign that training was unstable. An improved recipe was then tried on Google Colab with a GPU. Four variants were compared on validation data only, and the test set was evaluated once, on the 224 px variant.



Figure 4. Validation macro-F1 per epoch. The dot marks the epoch whose weights were saved (highest validation score). Both curves are read from the saved training logs.

Validation comparison of the four variants (30 epochs each)

Variant	What was added	Best val macro-F1	Mean of last 5 epochs	Best val accuracy
r1	Lower learning rate with cosine decay	0.822	0.741	0.928
r2	r1 + stronger augmentation + label smoothing	0.781	0.761	0.891
r3	r2 + class-weighted loss	0.862	0.813	0.906
r4 (tested)	r3 at 224 px instead of 160 px	0.840	0.795	0.920

Values as printed by the Colab comparison step. The last-5 average shows how stable training is. Differences of a few hundredths between variants are within the noise of a 138-photo validation set.

An honest note on selection. The test run used the 224 px variant (r4). The 160 px variant with class weighting (r3) scored slightly higher on validation (0.862 versus 0.840) but was not tested; testing both and picking the better one by test score would have used the test set for selection. The saved r4 weights are from epoch 6, the highest validation score of the run (0.840). Later epochs settled between 0.78 and 0.81, so that validation figure is probably a slightly lucky peak. Finally, the first baseline's test score was seen before the improvements were designed.

9.2 Test-set results (scored on 138 photos kept aside; training used 1,122)

Model	Accuracy	Macro precision	Macro recall	Macro-F1
First baseline (160 px, basic training)	76.8%	0.818	0.823	0.791
Improved baseline (224 px, tuned training)	84.1%	0.732	0.756	0.729

How to read these numbers. Accuracy rose from 76.8% to 84.1%, but macro-F1 fell from 0.791 to 0.729. Both are real and they do not contradict each other. Macro-F1 gives each of the 16 classes equal weight, and four classes have a single test photo. The improved model missed two of those single photos (anthracnose and tomato spotted wilt virus), each scoring 0 and together costing about 0.12 of macro-F1. With 138 photos, an accuracy of 84.1% carries roughly +/-6 points of uncertainty (95% interval), and the earlier 76.8% roughly +/-7. The two accuracies therefore overlap somewhat; the gain is likely real but its size is uncertain.

9.3 Results by disease

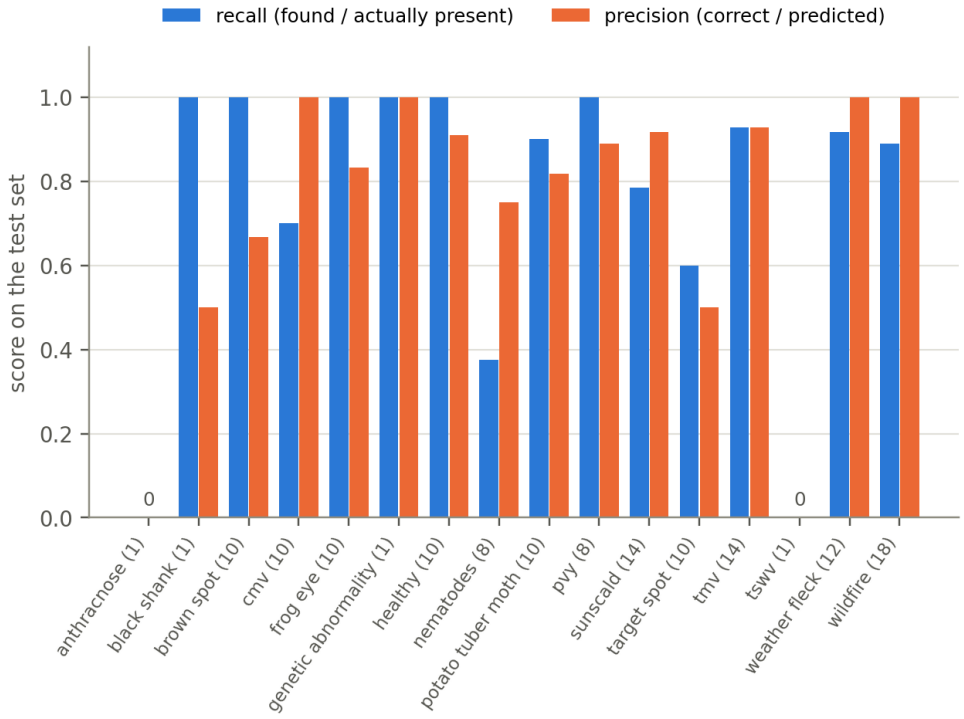


Figure 5. Test-set recall and precision of the improved baseline per class. The number in brackets is the class's test photo count. Anthracnose and TSWV scored 0 (one photo each).

Class	Test photos	Precision	Recall	F1
anthracnose	1	0.00	0.00	0.00
black shank	1	0.50	1.00	0.67
brown spot	10	0.67	1.00	0.80
cmv	10	1.00	0.70	0.82
frog eye	10	0.83	1.00	0.91
genetic abnormality	1	1.00	1.00	1.00
healthy	10	0.91	1.00	0.95
nematodes	8	0.75	0.38	0.50
potato tuber moth	10	0.82	0.90	0.86
pvy	8	0.89	1.00	0.94
sunscald	14	0.92	0.79	0.85
target spot	10	0.50	0.60	0.55
tmv	14	0.93	0.93	0.93
tswv	1	0.00	0.00	0.00
weather fleck	12	1.00	0.92	0.96
wildfire	18	1.00	0.89	0.94

- **Strongest:** pvy, frog eye, brown spot, healthy and genetic abnormality all reach recall 1.0 (genetic abnormality on a single photo).
- **Weakest with several photos:** nematodes (recall 0.38), target spot (0.60) and sunscald (0.79).
- **Over-predicted:** brown spot, black shank and target spot have precision of 0.67, 0.50 and 0.50, so other diseases are sometimes labelled as these.

9.4 The deployed model was retrained on all photos

After the evaluation above, the final model for the website was retrained on **all 1,398 real photos** (train, validation and test) with the same recipe, 30 epochs, keeping the last epoch. More data can help, especially for rare classes. The cost is that **no unseen photos remain, so this model has no measured score**. Scoring it on the old test photos would be meaningless because it has seen them (it scored 100% on them, which says nothing about new photos). The 84.1% / 0.729 figures are the best available estimate for this recipe, not a measurement of the deployed model. Training loss fell from 1.81 to 0.76, as expected for this setting.

10. Results: the federated experiments

These are the project's central results. Each of the four experiments was run **once**, with the same data split, starting weights and seed (42). The test set was scored once per run, on the final shared model after round 20 (not on the best round).

10.1 How the runs were made

Experiments ran on Google Colab, on its CPU (about 13 to 14 minutes each, 20 rounds, 8 simulated clients with 4 chosen per round, uneven split with Dirichlet alpha 0.5). The Colab GPU could not be used: client processes crashed inside PyTorch's CUDA dropout step on the image available, so a CPU switch was added. Settings were not changed after seeing any results. The two experiment-specific rules are in section 5.

10.2 Summary table

	Selection	Compression	Test accuracy (95% interval)	Macro-F1	Rounds to converge	Rounds to target	Upload (MB)	Total (MB)
A	random	none	81.9% (75-87%)	0.724	16	14	729	1,458
B	adaptive	none	84.8% (78-90%)	0.688	20	6	729	1,458
C	random	topk	80.4% (73-86%)	0.717	16	14	155	884
D	adaptive	topk	81.9% (75-87%)	0.663	18	14	155	884

Test set: 138 photos. The 95% interval for accuracy is a binomial (Wilson) interval and shows how much one test set can move a score. "Rounds to converge" is the first round within 1 point of the run's own best validation accuracy. "Rounds to common target" is the first round reaching 77.1% validation accuracy, 95% of the lowest best value among the four runs.

10.3 Figures

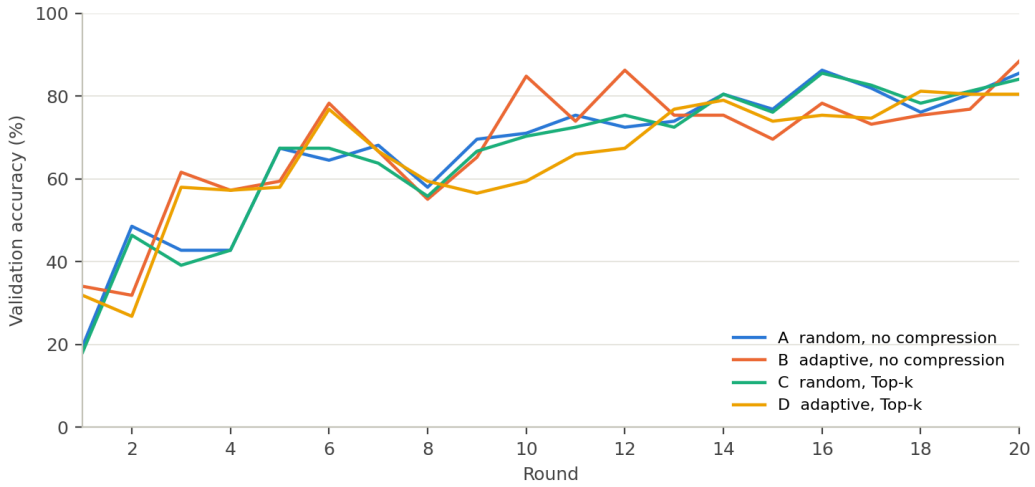


Figure 6. Validation accuracy of the shared model after each round. The curves jump from round to round because only 4 of 8 clients train each round and each client holds an uneven mix of diseases.

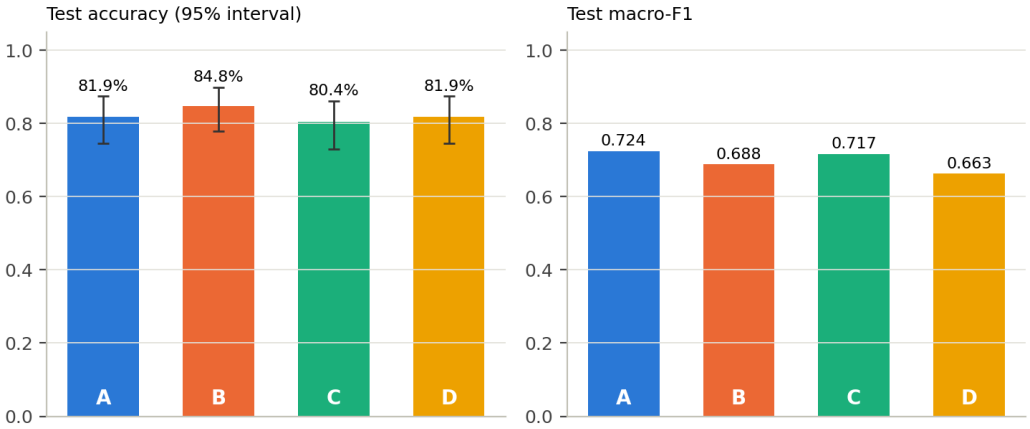


Figure 7. Test accuracy (with 95% interval) and macro-F1 of the final shared model. The intervals of all four runs overlap almost completely.

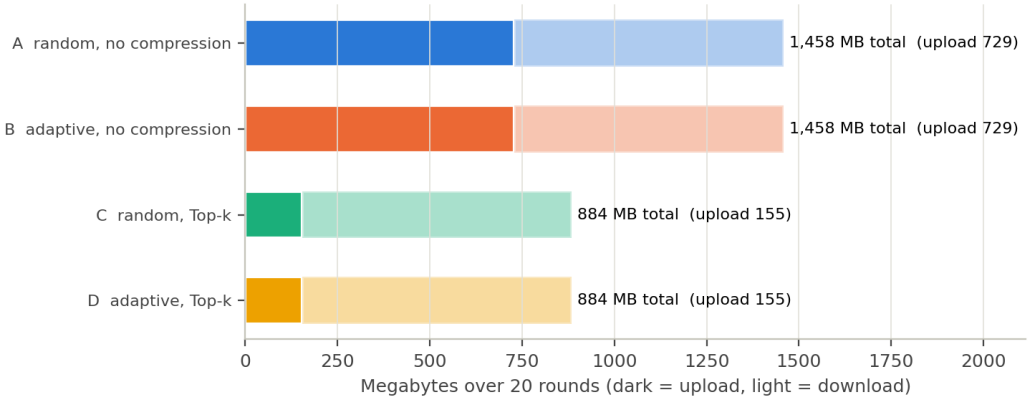


Figure 8. Communication over 20 rounds. Dark = upload from clients, light = download to clients. Compression shrinks the upload only.

10.4 What the results support

- **Top-k compression cuts the upload by 79%** (729 MB to 155 MB), with random selection (A to C) and with adaptive selection (B to D). This is a count of bytes, not an estimate, so it is the one result that does not depend on the noisy test set.
- **Total traffic falls by 39%, not 79%,** because the server still sends the full model to every selected client each round (729 MB of 1,458 MB). Compressing the download is a natural next step.
- **Accuracy is not clearly changed by compression.** 81.9% (A) versus 80.4% (C), and 84.8% (B) versus 81.9% (D). All differences are smaller than the width of the test-set interval (about 12 points).
- **Selection does not change bytes per round,** because the same number of clients is chosen. A and B, and C and D, therefore send exactly the same amount. A benefit of adaptive selection could only appear as faster convergence or better accuracy.

10.5 What they do not support

- **No claim that adaptive selection helps or hurts.** B has the highest accuracy but reached its best validation accuracy only at round 20; D matches A on accuracy. In B, the common validation target was reached at round 6 against 14 for A, but that rests on one validation round (round 6) that was followed by a drop to 55% at round 8, and D shows no such speed-up (14). With one run each, this cannot be told apart from luck.
- **The lower macro-F1 of B and D is mostly one photo.** There is a single black shank photo in the test set (and only 7 training photos). A and C classified it correctly (F1 1.00); B and D did not (F1 0.00), which alone is worth 1/16 = 0.0625 of macro-F1. Averaged over only the 12 classes with 8 or more test photos, macro-F1 is A 0.799, B 0.834, C 0.789, D 0.800, so the apparent gap shrinks or reverses. This is a

description of what one photo did, not evidence that adaptive selection is better.

- **One seed.** A different seed changes the data split between clients, the random choices and the order of training. Repeating all four experiments with several seeds and reporting the spread is needed before a paper can claim a difference between methods.
- **Not comparable with the baseline.** The federated runs used basic training at 160 px; the baseline used a tuned recipe at 224 px. The numbers should not be compared across sections 9 and 10.

10.6 A bug found and fixed along the way (experiment C)

The first run of experiment C ended with 0.7% accuracy: the shared model became NaN (not a number) at round 13 and stayed broken. The clients' training loss stayed normal, which points at the BatchNorm running statistics: they are used only when the model is evaluated, and the first version sent them through Top-k like gradients. A variance cannot be negative, but a partly-sent, error-corrected update can push one below zero, which makes the evaluation produce NaN. This explanation fits all the evidence but was not tested in isolation. The fix was to compress only the learned weights and send the BatchNorm statistics in full, and to stop a run immediately if the shared model ever contains NaN. Experiment C was rerun and finished normally. Experiments A and B use no compression and were unaffected. **Consequence for interpretation:** in C and D the 10% rule applies to the learned weights, and the small BatchNorm statistics are sent uncompressed. The first, broken run is not used anywhere in this report.

10.7 Results by disease (test F1)

Class	Test photos	A	B	C	D
anthracnose	1	0.00	0.00	0.00	0.00
black shank	1	1.00	0.00	1.00	0.00
brown spot	10	0.67	0.82	0.67	0.89
cmv	10	0.80	0.83	0.80	0.77
frog eye	10	1.00	0.91	1.00	0.83
genetic abnormality	1	1.00	1.00	1.00	1.00
healthy	10	0.91	0.90	0.91	0.90
nematodes	8	0.57	0.67	0.57	0.67
potato tuber moth	10	0.80	0.80	0.84	0.80
pvy	8	0.55	0.57	0.55	0.33
sunscald	14	0.83	0.79	0.73	0.83
target spot	10	0.64	0.82	0.58	0.84
tmv	14	0.92	0.93	0.92	0.93
tswv	1	0.00	0.00	0.00	0.00
weather fleck	12	1.00	1.00	1.00	0.96
wildfire	18	0.90	0.97	0.90	0.86

Classes with 1 test photo score either 0 or 1 and say almost nothing. Anthracnose and tswv scored 0 in all four runs.

10.8 Client participation

Client	Photos	Times chosen, A	Times chosen, B	Times chosen, C	Times chosen, D
0	111	12	10	12	11
1	171	8	8	8	8
2	176	12	8	12	8
3	105	8	10	8	10
4	53	10	13	10	13
5	92	12	12	12	12
6	131	8	11	8	11
7	283	10	8	10	7

Each client could be chosen at most 20 times. Random selection chose clients 8 to 12 times; adaptive selection 7 to 13 times. Adaptive selection did not concentrate training on a few clients here.

11. Reliability: what the numbers can and cannot support

- **Small test set.** 138 photos. Sixteen classes, four of them with one test photo. One photo moves a rare class from 0 to 1 and macro-F1 by about 0.06. The black shank photo alone separated A and C from B and D (section 10.5).
- **One split and one seed.** All scores come from one training, validation and test split, and each federated experiment was run once. Repeated cross-validation and several seeds would give a much more trustworthy estimate and error bar. Neither has been done.
- **Selection on a small validation set.** Choosing the best epoch and the best variant from the same 138-photo validation set makes validation figures optimistic. That is why the test set is the headline.
- **Confidence is not trustworthy as a guide to correctness.** A wrong answer can carry 50% to 90% confidence, so a confidence threshold cannot reliably catch errors.
- **The model always answers.** It picks one of 16 classes even for something that is not a tobacco leaf.
- **What is solid.** The byte counts in the communication results are exact. The accuracy and F1 differences between experiments are not.

12. Limitations

12.1 Photos from other sources

This is the most important limitation found so far. Three photos supplied by the project owner from other sources were classified wrongly by the improved baseline model (before the final retraining):

Photo label (as given)	Meaning	Model answer	Confidence
Early_FLS	Early frogeye leaf spot (class frog eye)	Weather fleck	40.5%
cercni	Cercospora nicotianae, frogeye (class frog eye)	Weather fleck	54.6%
weather fleck	Weather fleck	Genetic abnormality (weather fleck second, 25.5%)	52.8%

Three photos are far too few to measure accuracy and the labels were given by the owner, not independently verified. They do show the model failing on every outside photo tried, in contrast to about 83% to 84% on the dataset's own test photos. The likely explanation is that outside photos look different from the training photos (camera, lighting, close-up crops, early-stage lesions) but this cause is not proven. A first suspect, the prediction service's colour correction, was ruled out as the main cause. The final model was not re-checked on these photos. Until a larger outside test is done, the honest claim is: the model works on photos like those in its dataset; behaviour on other photos is unknown and so far poor.

12.2 Other limitations

- **Federated results are simulated and single-seed.** All clients run on one machine. Communication is measured in bytes sent, not network time, and there are no real network faults or device limits. Each experiment ran once.
- **The federated settings are not the tuned ones.** They use basic training at 160 px, so absolute accuracies are not those the best model could reach.
- **Rare classes.** Four classes have 7 to 18 real training photos. Whatever a federated or centralised model learns for them is fragile.
- **The deployed model has no score of its own** (section 9.4).
- **Dataset licence.** No explicit licence is stated; academic use with citation is assumed and the images cannot be redistributed. The repository contains the trained model file but not the images.
- **Not a diagnosis.** The output is an estimate from photographs. Real crop problems need an agronomist.
- **Disease descriptions are unreviewed.** They are general background and have not been checked by a plant-pathology expert.
- **Hosted prediction not verified on Vercel** (build, model loading, start-up time and the time limit on the free plan).
- **GPU runs did not work on the Colab image used.** Experiments ran on CPU; the GPU path remains unverified.
- **Version control.** The code is in a GitHub repository, but the latest changes (new model, pages, federated fixes, this report) are not yet committed.

13. Completion status

States: Done means built and checked as described in section 8; Partly means some of it exists; Not started means no work yet.

#	Work item	State	What is left
1	Dataset audit, split and leakage controls	Done	Optional: repeated cross-validation for rare classes
2	Web application pages and design	Done	Visual review in a browser by a second person; check the About page text against the results
3	Authentication and Firestore rules	Done	Confirm rules are published to the live Firebase project
4	Federated-learning code and tests	Done	GPU path unverified
5	Export, compare and import tools	Done	Run the import with the real results
6	Centralised baseline model	Done	Optional: test the r3 variant and report both
7	Prediction service and diagnosis page	Done	Verify the hosted function on Vercel
8	Federated experiments A to D	Done (1 seed)	Extra seeds to support any claim about methods
9	Real results shown in the web app	Not started	Import, then check charts and tables
10	Tests on photos from other sources	Not started	Collect labelled outside photos and measure
11	Deployment	Partly	Confirm the live site, authorised domains and the hosted prediction
12	Final report and version control	Partly	Commit the latest changes; add extra seeds if run

Reading the status. 8 items are done, 2 are partly done and 2 are not started. The core research experiments have now been run once each; what is missing is repeating them to make claims about the two methods, showing the results in the web app, and the check on outside photos.

14. Future scope and remaining work

14.1 Required to complete the project (in order)

- **Import the real results into Firebase** using the owner's account and a service-account key stored outside the repository, then check every chart, table, the confusion matrix and the participation view with real data.
- **Check the About page's text** against the final results (the landing page status was already updated).
- **Run extra seeds** for all four experiments (a few seeds, with the spread reported) so that differences between methods are not mistaken for luck.
- **Test on outside photos.** Collect a few dozen labelled photos from other sources, measure per-class accuracy and decide whether retraining on more varied data is needed.
- **Verify the live deployment**: the hosted prediction function, the Firebase authorised domain and the published rules.
- **Expert review** of the disease descriptions and class names.
- **Version control and final write-up** including limitations and dataset credits.

14.2 Improvements that would strengthen the work

- **Compress the download too.** It is now more than half of the traffic once uploads are compressed.
- **Understand adaptive selection.** Test whether it helps convergence speed over several seeds, and try other score weights chosen on validation data only.
- **Other ways to handle BatchNorm** in federated training, and comparison with methods designed for uneven client data.
- **More reliable scoring.** Repeated stratified cross-validation, reported with error bars, instead of a single 138-photo test set.
- **More data for rare classes and for outside conditions.** The full 2,721-image source collection (by contacting its authors) or the Roboflow frogeye and brown-spot images (CC BY 4.0) with proper credit, split by original photo so duplicate or augmented copies never straddle training and test.
- **A "not a tobacco leaf / unsure" outcome** so the model can decline to answer, and better-calibrated confidence.
- **Other compressors** such as quantisation, and privacy extras such as secure aggregation or differential privacy, which federated learning alone does not provide.
- **Real-device trials** with phones or edge devices to measure true network and energy cost.
- **Usability work** such as local-language text, a mobile-friendly capture guide for good photos, and feedback from growers.
- **Hosting and monitoring** of the prediction function, including usage limits.

These are suggestions, not commitments; none has been started.

15. How to run and reproduce

These commands are from the project's own scripts. Detailed set-up is in the repository's README files and the docs folder.

Task	Command
Website (from the web folder)	npm run dev
Rule tests and import test	npm run test:rules and npm run test:import
Python tests (from the ml folder)	python -m pytest
Train the centralised baseline	python -m src.training.centralised --epochs 30 --lr 0.005 --cosine --strong-aug --label-smoothing 0.1 --balance --img-size 224 --skip-test --out <name>
Final model on all photos	the same command with --train-on-all instead of --skip-test (no score is possible afterwards)
Test a chosen run once	python -m src.training.final_test --out <name>
Run one federated experiment	python -m src.federated.run_experiment --exp A (add --seed N for another seed, --cpu to avoid the GPU)
Run all four, then compare	python -m src.federated.run_all, then python -m src.export.compare --seed 42
Federated runs on Colab	ml/colab/federated_on_colab.ipynb (the notebook for the baseline is ml/colab/train_on_colab.ipynb)
Import results into Firebase	python -m src.export.import_to_firestore (with the owner's user id and service-account key; --dry-run to preview)
Export the model for the hosted function	python -m src.serve.export_onnx --model <name>
Start the local prediction service	set MODEL_DIR=<name> then python -m src.serve.app

Configuration lives in ml/configs/base.yaml, which drives all four experiments so they share data, model and training settings. Secrets are never committed: .env* files, service-account keys and the PyTorch model files are ignored by git.

Conventions followed

- No metrics, predictions or dataset statistics were invented; every number in this report is read from saved results or stated as given.
- No training code in the website, and none on the hosting platform.
- Authorisation is enforced by database rules, not only by the interface.
- Anything that was not tested is labelled as such.

16. References and credits

- Lin et al., Tobacco Leaf Abnormality dataset papers: SPIE International Conference on Digital Image Processing (ICDIP) 2022, and Frontiers in Plant Science 2024. Dataset: Hugging Face TamAko783/TPDD_Honglin_CLS.
- McMahan, Moore, Ramage, Hampson and Aguera y Arcas, Communication-Efficient Learning of Deep Networks from Decentralized Data, AISTATS 2017 (FedAvg).
- Sandler, Howard, Zhu, Zhmoginov and Chen, MobileNetV2: Inverted Residuals and Linear Bottlenecks, CVPR 2018.
- Beutel et al., Flower: A Friendly Federated Learning Framework, 2020 (arXiv:2007.14390).
- Lin, Han, Mao, Wang and Dally, Deep Gradient Compression, ICLR 2018; Stich, Cordonnier and Jaggi, Sparsified SGD with Memory, NeurIPS 2018 (Top-k with error feedback).
- Finlayson and Trezzi, Shades of Gray and Colour Constancy, Color and Imaging Conference, 2004.
- Kunyadini, Tobacco leaf disease detection, Roboflow Universe, CC BY 4.0 (considered, not used for training).

Reference details are given from memory of the standard citations and should be checked against the originals before submission.

Appendix A. Glossary

Term	Plain meaning
Federated learning	Training a shared model while the training data stays with its owners; only model updates travel
FedAvg	The standard way to combine client updates: a weighted average, weighted by each client's number of photos
Client / round	A data owner taking part (here a simulated farm); one cycle of select, train, upload, combine
Uplink / downlink	Data sent from clients to the server / from the server to clients
Non-IID	Clients hold different kinds of data (for example different diseases), not a random mix
Top-k compression	Sending only the k largest entries of an update, with their positions
Error feedback	Remembering the entries not sent and adding them to the next update
BatchNorm statistics	Running averages kept inside the network that stabilise it; they are not learned by gradients, so they are not compressed
Gradient / update	The change a client makes to the model during local training
Seed	A number that fixes the random choices of a run, so it can be repeated; different seeds give different splits and draws
Accuracy	Share of photos classified correctly
Precision	Of the photos the model labelled as class X, the share that really are X
Recall	Of the photos that really are X, the share the model found
F1 and macro-F1	F1 balances precision and recall for one class; macro-F1 averages F1 over all classes equally
95% interval	A range that would contain the true accuracy in about 95 of 100 similar test sets; wide for small test sets
Confusion matrix	A table of true class against predicted class that shows which diseases are mixed up
Validation vs test set	Validation guides choices during development; the test set is kept aside for one final score
Leakage	When near-copies of training photos appear in test data, making scores look better than they are
Overfitting	A model memorising its training photos instead of learning general patterns
Epoch	One full pass over the training photos
Smoke test	A tiny run that only proves the pipeline works; its numbers mean nothing
ONNX	A portable file format for trained models, used so the website can run the model without PyTorch
Shades-of-Gray	A colour-balancing method that removes the colour cast of lighting

Appendix B. The 16 classes in plain language

General background written for readers who are not plant-pathology specialists. These descriptions have not been reviewed by an expert and are not treatment advice.

Class (model label)	Cause	In brief
Healthy leaf (healthy)	None	No disease or damage; even green colour
Anthracnose (anthracnose)	Fungus (Colletotrichum)	Sunken or dry brown to tan lesions, sometimes with darker edges
Black shank (black_shank)	Water mould (Phytophthora nicotianae)	Soil-borne; yellowing and wilting, stem base turns black
Brown spot (brown_spot)	Fungus (Alternaria)	Round brown spots, often with target-like rings, mostly on older leaves
Cucumber mosaic virus (cmv)	Virus	Light and dark green mottling, puckered or narrow leaves; spread by aphids
Frogeye leaf spot (frog_eye)	Fungus (Cercospora nicotianae)	Round spots with a pale centre and dark border
Genetic abnormality (genetic_abnormality)	Genetic	Unusual leaf shape, colour or pattern not caused by a pathogen
Nematode damage (nematodes)	Pest (root-knot nematodes)	Stunting and patchy yellowing; the roots are affected
Potato tuber moth damage (potato_tuber_moth)	Pest (caterpillar)	Winding tunnels or transparent blotches inside the leaf
Potato virus Y (pvy)	Virus	Mottling, dark or dead veins; spread by aphids
Sunscald (sunscald)	Environment	Bleached, papery patches from intense sun
Target spot (target_spot)	Fungus (Rhizoctonia solani)	Brown spots with rings; the dead centre may fall out leaving holes
Tobacco mosaic virus (tmv)	Virus	Mosaic pattern, curled or distorted leaves; spreads by touch and tools
Tomato spotted wilt virus (tswv)	Virus	Brown or bronze ring spots and dead patches; spread by thrips
Weather fleck (weather_fleck)	Environment (ozone)	Many small white, tan or grey flecks on upper sides of older leaves
Wildfire (wildfire)	Bacterium (Pseudomonas syringae pv. tabaci)	Brown spots with a wide yellow halo, spreads in wet windy weather